Raspberry Pi 5 + AI HAT (Hailo-8L)

The gateway tier — measured, and two assumptions refuted

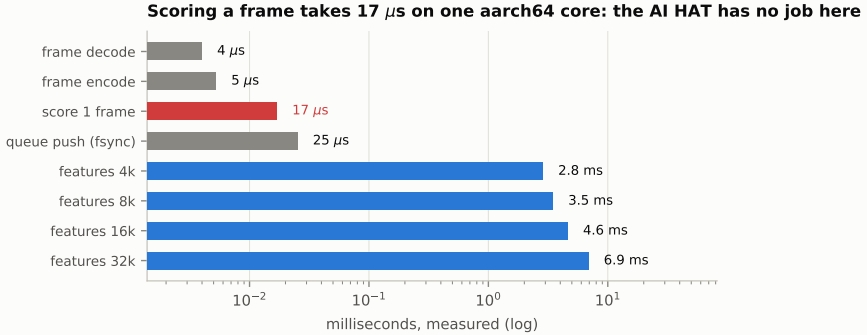
Compute	4× Cortex-A76 2.4 GHz, 8 GB
Accelerator	Hailo-8L, 13 TOPS
Power	7.5 W sustained, mains
Score a frame	**17 μs measured**
Fan-out	**781 331 hourly nodes on one core**
Cost	∼EUR 200 and it carries a site

The only figures in this repository from a **stopwatch** rather than a cycle count — the gateway tier runs on general-purpose aarch64 Linux, and so did the measurement.

It was not a Pi 5. The benchmark says so in its own output and stores the caveat in its JSON, because a throughput number without its machine is unquotable.

Branch `hw/rpi5-hailo-gateway`. 18 gateway tests, 36 shared edge-agent tests. Figures generated from `results/benchmark.json` and `results/holdover.json`.

What a gateway actually costs, measured

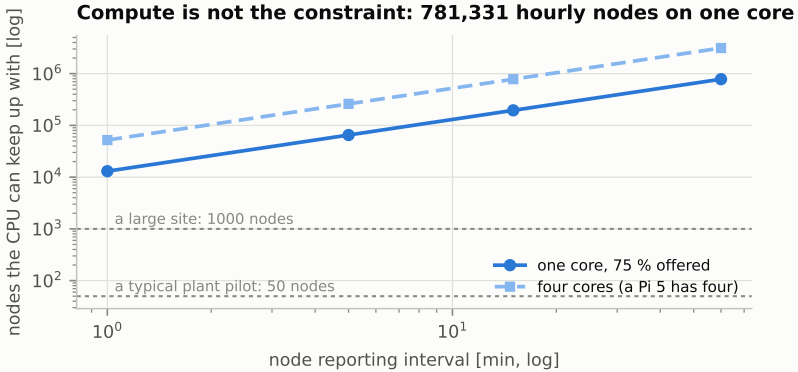


Five orders of magnitude, and the two stages an accelerator could touch are at the bottom of it. Scoring is **17 µs**; the feature extractor is 200× that; the acquisition it serves is 100× the extractor.

A measurement that measured nothing, and says so.
The durable queue reported 39 828 frames/s with `synchronous=FULL` — implausible. The benchmark detects that `durable` and `nosync` agree within 20 % and warns that the `fsync` is not reaching storage on this host.

A benchmark that silently measures nothing is worse than none, because the number gets quoted.

Refuted assumption 1: compute is not the constraint



One core, 75 % offered, keeps up with **781 331** hourly nodes. A large site has a thousand. Three orders of magnitude of headroom.

If compute were the reason for a gateway tier, a gateway tier would not be needed.

What *does* bind a gateway — none of which this benchmark measures:

the broker's concurrent connections and memory per connection · the time-series database's write rate · the uplink · and **BLE airtime**, a shared medium with a hundred nodes waking on their own schedules.

Measuring them needs the compose stack running, and there is no Docker in this environment.

Pi 5 gateway

So what is a gateway for?

1. Protocol translation. A BLE node cannot reach a cloud broker, and the thing that terminates the radio is the only mains-powered element in the chain anyway.

2. Site-scale store-and-forward. A node's flash holds days; a gateway's SSD holds months.

3. Local autonomy. An alarm needing a cloud round trip is unavailable during the outage that most often *accompanies* a real failure. The gateway scores locally and alarms with no uplink at all.

4. Knowing which nodes have gone quiet.

(4) is the one that cannot live anywhere else.

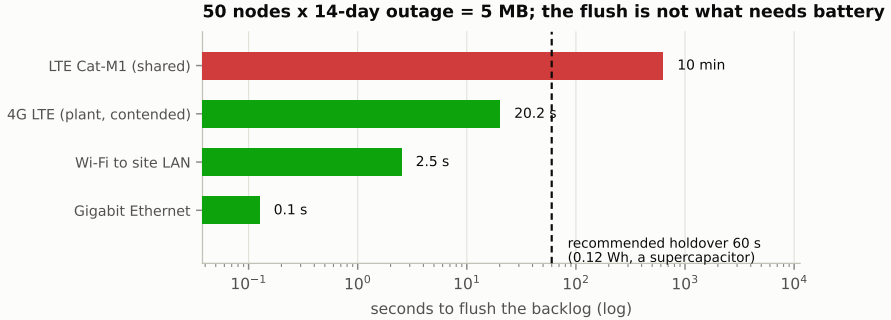
A node that stops reporting is the commonest real event in a deployed fleet, and downstream it is **indistinguishable from a healthy quiet machine** — both produce no data. Only the thing the nodes report *to* knows the difference.

Three decisions, each with a failure behind it:

- interval **learned**, not configured — fleet config drifts;
- learned **slowly** (EWMA 0.8/0.2) — one early frame must not raise the threshold and mask the outage that follows;
- **three** missed intervals, not one — crying wolf trains the operator to ignore it.

And silence is measured from **arrival**, not the frame's timestamp: a node with a wrong clock would look permanently fresh.

Refuted assumption 2: holdover is not the correctness requirement



This kit's own plan said "UPS holdover is a correctness requirement". It is not. The backlog is already safe — SQLite `synchronous=FULL` fsyncs every frame — and the flush happens when mains *and* uplink are back, not on battery.

So the UPS buys a graceful SQLite close (30 s is generous) and brownout ride-through. **60 s of supercapacitor, 0.12 Wh** — no battery chemistry in the enclosure, no maintenance item.

The correction that matters more: storage.
If the SD card dies on an unclean power-down, the site's backlog is gone whatever the holdover. An NVMe SSD or eMMC carrier is a larger data-integrity win *and* costs less than the UPS it partly replaces.

The queue fsyncs once per frame: 1 200 writes/day at 50 nodes hourly is trivial; 1 Hz aggregation across a large site would consume an SD card in months.

The AI HAT: what it would take to earn its place

	Latency
63-float detector, numpy on one CPU core	**17 μs**
1D CNN over 8192 raw samples, $\sim$4 MMAC, on CPU	$\sim$8 ms
the same CNN on a Hailo-8L	$\sim$0.3 ms
. . . for 200 channels at once	still $\sim$0.3 ms

The HAT is justified by a **model class**, not a speed-up: a raw waveform model skips the feature extractor entirely.

That is the appeal — every feature here is a human decision about what matters, taken in advance — and the risk: a waveform model can learn the test rig's mains hum and call it a bearing fault.

`HailoScorer` **raises on construction**, and the exception lists what is missing:

1. a 1D CNN trained on *measured* data;
2. ONNX $\rightarrow$ `.hef`, int8-calibrated on real waveforms;
3. a bit-exactness check against the ONNX reference;
4. decision-flip accounting against the float model.

A stub returning plausible probabilities would make this kit look finished while shipping confident scores from a nonexistent model.

So what: Buy the Pi, skip the HAT, spend the EUR 70 on the NVMe SSD — where it buys data integrity.

Status, and what is deliberately absent

Fan-in, forwarding, queueing	18 tests
Gap detection, wrap-safe	✓
Local alarms during an outage	✓
Silent nodes, learned intervals	✓
Corrupt frame rejected	✓
Bad scorer does not stop forwarding	✓
Throughput measured	this host
Run on a Pi 5	no
`fsync` cost measured	no
Compose stack started	no
Hailo-8L used	by design, no
Radio termination	not written

The radio side does not exist, deliberately. `Gateway.handle()` takes bytes. What delivers them — a BLE GATT server, a sub-GHz concentrator, a local broker — is the one part that genuinely depends on the radio hardware chosen, and stubbing it would hide the choice.

The undecided question a review should force: may the gateway read node payloads? If it can, compromising one gateway compromises a site. If it cannot, local scoring — reason (3) for having a gateway at all — stops working. That trade should be decided deliberately, and it has not been.